ИНФОРМАЦИОННАЯ СИСТЕМА ВУЗА

единое окно студента

технологический стек

Один движок заявок с конфигурируемыми типами, модули ячеек выдачи, бронирования и приёмных слотов. Единый бэкенд, три клиента, один контракт OpenAPI.

01 **Архитектура**

02 **Бэкенд**

03 **Клиенты**

04 **Функционал**

05 **Репозиторий и готовность**

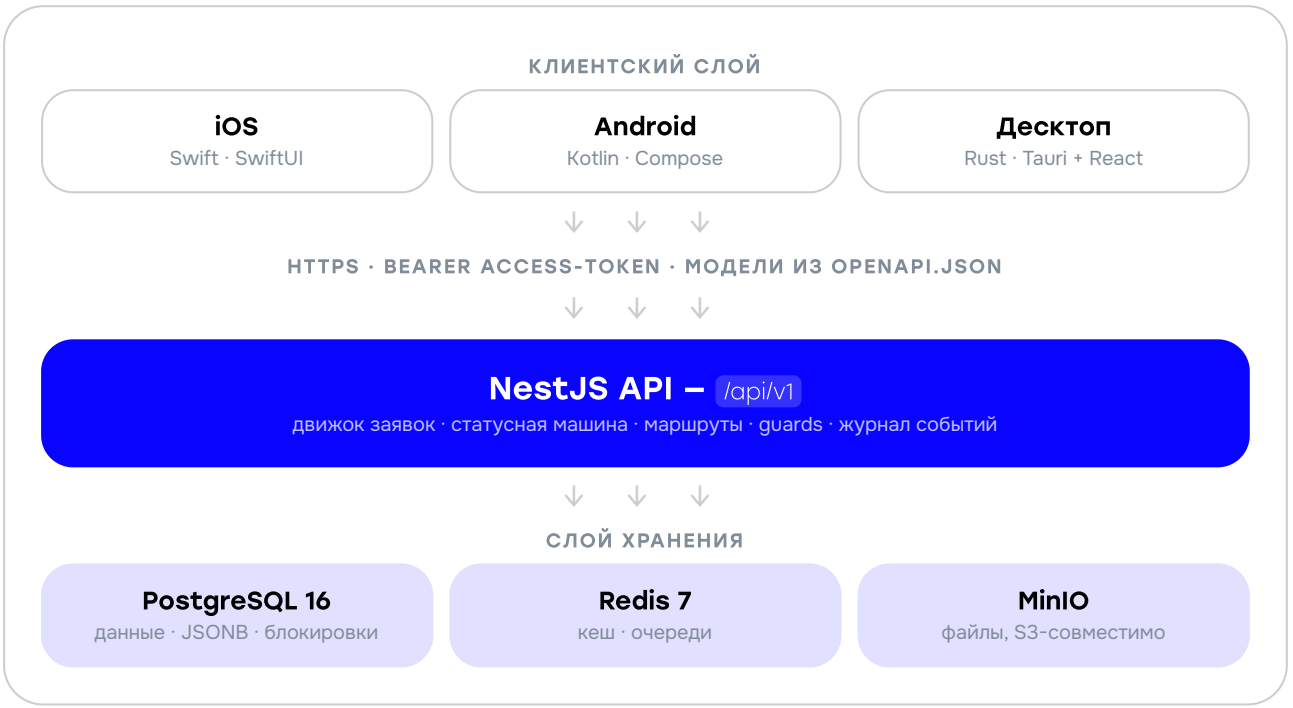
TypeScript · Rust · Swift · Kotlin · PostgreSQL

ВЕРСИЯ ДОКУМЕНТА 1.0

9 сентября 2026 г.

архитектура

Клиенты не содержат бизнес-логики: каждый эндпоинт проверяет права на сервере. Модели клиентов генерируются из одного `openapi.json`, поэтому контракт не может разъехаться между платформами.



packages/shared – общий источник правды

Типы и перечисления, статусная машина `ALLOWED_TRANSITIONS`, матрица прав, Zod-схемы `fields_schema` и `access_rule`, генератор формы `buildFormSchema`. Подключается напрямую бэкендом и десктопом; для iOS и Android та же логика портирована, а контракт данных приходит из OpenAPI.

4	3	1
языка в проекте	клиентских приложения	контракт OpenAPI

бэкенд

`apps/api` — реализован строго по техническому заданию, раздел 2.1.

СЛОЙ	ТЕХНОЛОГИЯ	ЗАЧЕМ
Язык	TypeScript 5.6	Общие типы с фронтом, один язык на проект
Рантайм	Node.js 20 LTS	Стабильность
Фреймворк	NestJS 10	Модульность, DI, guards и pipes ложатся на задачу
ORM	Prisma 5	Декларативная схема, миграции, типобезопасность
База данных	PostgreSQL 16	JSONB для схем полей, транзакции с блокировками
Кеш и очередь	Redis 7	Сессии, фоновые задачи
Планировщик	@nestjs/schedule	Cron: просроченные ячейки, напоминания по SLA
Файлы	MinIO (S3)	Драйвер сменный через DI, локально в Docker
Аутентификация	JWT access + refresh	Access 15 мин, refresh 30 дней, ротация
Пароли	argon2id	—
Валидация	Zod 3	Одна схема на сервер и клиент
Генерация PDF	Puppeteer	HTML-шаблон → A4, поля 20 мм
Документация API	Swagger / OpenAPI	Автогенерация из декораторов
Тесты	Vitest + Supertest	Юнит и интеграционные
Инфраструктура	Docker Compose	Postgres + Redis + MinIO одной командой
CI	GitHub Actions	lint → тесты → миграции → сборка

Безопасность

Права — только на сервере. `RolesGuard` по роли, `ApplicationAccessGuard` по владению заявкой или участию подразделения в маршруте.

Файлы. Проверка расширения *и* магических байт, лимит размера, имя на диске — UUID, отдача только через контроллер.

Транзакции. Любая смена статуса — одна транзакция вместе с записью в журнал событий и созданием уведомления.

Журнал неизменяем. Роль приложения лишена UPDATE и DELETE на таблице `event_log`.

клиенты

ДЕСКТОП · РАБОЧЕЕ МЕСТО СОТРУДНИКА

Tauri 2 + React

Оболочка на Rust, интерфейс на TypeScript. Установщики `.msi` и `.dmg`, автообновление, печать PDF через системный диалог, refresh-токен в защищённом хранилище ОС. Для тех, кто оформляет справки: широкие таблицы, мышь и клавиатура.

СЛОЙ	ТЕХНОЛОГИЯ
Оболочка	Tauri 2 — ядро на Rust
Интерфейс	React 18 + TypeScript + Vite 5
Роутинг и данные	React Router 6, TanStack Query 5
Формы	React Hook Form + Zod
Стили	Tailwind CSS, токены PCO

МОБИЛЬНЫЙ КЛИЕНТ

iOS

Язык	Swift 5.9
UI	SwiftUI, цель iOS 16
Сеть	swift-openapi-generator
Токены	Keychain
Пуши	APNs
Публикация	TestFlight → App Store

МОБИЛЬНЫЙ КЛИЕНТ

Android

Язык	Kotlin
UI	Jetpack Compose, minSdk 26
Сеть	Retrofit + kotlinx.serialization
Токены	EncryptedSharedPreferences
Пуши	Firebase Cloud Messaging
Публикация	Internal track → Google Play

Дизайн-система PCO

Общая для всех трёх клиентов. Палитра из четырёх значений, контурные карточки без единой тени, трекинг ноль, движение 120–200 мс только по прозрачности и цвету.

`#0804FF синий` `#FFFFFF белый` `#EEEEEE серый` `#000000 чёрный` `Actay Wide — титульный` `Stolzl — заголовки и числа` `Onest — текст`

функционал

Ключевой принцип. Система — это один движок заявок с конфигурируемыми типами, а не набор независимых модулей. Тип заявки — это запись конфигурации: схема полей, маршрут согласования, правило доступа, флаги спецмодулей. Новый тип добавляется через админку и работает без единой правки кода ядра.

Ядро заявок

- Динамическая форма по `fields_schema`
- Маршруты с копированием этапов при подаче
- Статусная машина из 10 статусов
- Возврат на доработку без пересогласования
- Автоэтапы по `access_rule`
- Журнал событий по каждому переходу

Модуль ячеек

- Поиск свободной ячейки `FOR UPDATE SKIP LOCKED`
- Код получения из шести цифр, срок хранения
- Подтверждение выдачи сотрудником
- Драйвер замка через DI — задел под электронные

Бронирование мест

- Блокировка строки периода — квота не превышается
- Лист ожидания с позициями в очереди
- Автоповышение при отмене чужой брони
- Запрет пересекающихся броней

Документы и подпись

- PDF из HTML-шаблона, A4, поля 20 мм
- Загрузка подписанного скана
- Реестр освобождённых с выгрузкой CSV

РАСШИРЕНИЯ СВЕРХ ТЕХНИЧЕСКОГО ЗАДАНИЯ

Обходной лист

Параллельные этапы маршрута: пять отделов согласуют одновременно, финальный этап открывается только когда закрыты все.

Объявления

Рассылка деканата на всех, курс или группу. Каждому адресату — уведомление и пуш.

Запись на приём

Приёмные окна с разбивкой на слоты, бронирование под блокировкой строки окна.

репозиторий

Монорепозиторий на pnpm workspaces. Нативные клиенты живут рядом и потребляют тот же контракт.

```
edinoe-okno/
├─ packages/shared/      TypeScript · типы, статусная машина, Zod-схемы
├─ apps/api/             NestJS · движок, модули, Prisma, seed
├─ clients/
│  ├─ desktop/           Tauri 2 (Rust) + React · рабочее место
│  ├─ ios/               Swift · SwiftUI
│  └─ android/           Kotlin · Jetpack Compose
├─ design/               шрифты и токены PCO, прототип
├─ docs/                 архитектура, API, деплой, ADR
└─ docker-compose.yml    Postgres · Redis · MinIO
```

Готовность

КОМПОНЕНТ	СОСТОЯНИЕ
Бэкенд	Все этапы ТЗ реализованы, сборка и типизация зелёные
Общий код	Готов, покрыт тестами
Десктоп	Все экраны собраны, фронт билдится; установщики — на машине с Rust
iOS	Каркас и план; сборка требует macOS и Xcode
Android	Каркас и план; сборка требует Android SDK

40	14	8	4
юнит-тестов	таблиц в модели данных	типов заявок в seed	роли пользователей

КРИТЕРИЙ КАЧЕСТВА АРХИТЕКТУРЫ

Новый тип заявки с собственными полями и маршрутом создаётся через админку и работает без единой правки кода ядра. Это главный аргумент на защите проекта.